

Java Collections Internals

ArrayList · HashMap · ConcurrentHashMap vs Hashtable — Interview Deep Dive

September 2026

Java Collections Internals: ArrayList, HashMap, ConcurrentHashMap vs Hashtable

1. ArrayList Internal Working
 - Basic structure
2. ArrayList add()
3. What Happens When Capacity Is Full?
4. Why Is add() O(1) If Resizing Is O(n)?
5. ArrayList.get(index)
6. ArrayList Insert in the Middle
7. Why ArrayList Is Usually Better Than LinkedList
8. ArrayList Interview Trap: Thread Safety
9. ArrayList Cheat Sheet
10. HashMap Internal Structure
11. HashMap put() — Step by Step
12. Why Power of Two?
13. Empty Bucket
14. What if Two Keys Go to the Same Bucket?
15. Why hashCode() Alone Is Not Enough
16. Collision Resolution
17. Why Red-Black Tree?
18. HashMap get()
19. HashMap Resize
20. What Happens During Resize?
21. Why Load Factor 0.75?
22. HashMap Complexity
23. HashMap Null Behavior
24. Why Must HashMap Keys Be Immutable?
25. Now: ConcurrentHashMap
26. Important: Java 7 vs Java 8+ ConcurrentHashMap
27. ConcurrentHashMap Internal Structure
28. How get() Works in ConcurrentHashMap
29. How put() Works in ConcurrentHashMap
30. Why CAS?
31. Does ConcurrentHashMap Use Locks?
32. Why ConcurrentHashMap Scales Better Than Hashtable
33. Hashtable Internal Working
34. Why Hashtable Is Slower Under Concurrency
35. Hashtable vs ConcurrentHashMap
36. Iterator Difference
 - Hashtable
 - ConcurrentHashMap
37. Why Doesn't ConcurrentHashMap Allow null ?
38. putIfAbsent() — Very Important
39. computeIfAbsent()
40. Thread-Safe Map Does Not Mean Thread-Safe Value
41. ConcurrentHashMap vs Collections.synchronizedMap()
42. HashMap vs ConcurrentHashMap
43. Hashtable vs HashMap
44. One Diagram You Should Memorize

ArrayList

HashMap

ConcurrentHashMap

45. Interview Scenario: HashMap

46. Interview Scenario: Why Is HashMap `get()` $O(1)$?

47. Interview Scenario: Why ConcurrentHashMap Over Hashtable?

48. The Most Important Traps

49. Final Interview Cheat Sheet

Java Collections Internals: `ArrayList`, `HashMap`, `ConcurrentHashMap` VS

`Hashtable`

This is a **high-value Java interview topic**. At your experience level, don't just know Big-O. You should be able to explain:

```
ArrayList
→ dynamic array + resizing

HashMap
→ hash → bucket → node → collision handling → treeification → resize

ConcurrentHashMap
→ HashMap-like structure + fine-grained concurrency

Hashtable
→ legacy synchronized hash table
```

I'll use the **Java 8+ implementation model**, which is also relevant to Java 17. Some exact implementation details are JDK-specific, so treat them as implementation details rather than API guarantees.

1. ArrayList Internal Working

Basic structure

`ArrayList` is backed by an array:



The current Java 17 implementation is explicitly described as a resizable-array implementation, with constant-time `get` / `set` and amortized constant-time `append`. Its growth policy is implementation-specific. ([GitHub](#))

2. ArrayList `add()`

Suppose:

```
List<String> list = new ArrayList<>();

list.add("A");
list.add("B");
list.add("C");
```

Internally:

```
elementData[]
  ↓
[A, B, C, null, null, ...]
```

There are two concepts:

```
size  
capacity
```

For example:

```
size    = 3  
capacity = 10
```

So:

```
“  
size = number of actual elements capacity = allocated array length
```

3. What Happens When Capacity Is Full?

Suppose:

```
capacity = 10  
size = 10
```

and you call:

```
list.add("K");
```

ArrayList needs a larger backing array.

Conceptually:

```
Old array  
[10 slots]  
↓  
Create larger array  
[~15 slots]  
↓  
Copy elements  
↓  
Use new array
```

The Java 17 implementation uses a growth calculation based on roughly **1.5× the old capacity** as the preferred growth amount, although the specification deliberately does not guarantee a particular growth factor. ([GitHub](#))

That's why people commonly say:

```
“  
ArrayList grows by roughly 50%.
```

But in an interview, phrase it as:

```
“  
“The common OpenJDK implementation grows capacity by roughly 1.5×; the API does not  
guarantee a specific growth factor.”
```

4. Why Is `add()` **O(1)** If Resizing Is **O(n)**?

Normal append:

```
0(1)
```

Occasionally resizing:

```
0(n)
```

But resizing happens infrequently.

Therefore the **amortized complexity** of append is:

```
0(1) amortized
```

This distinction is important.

5. `ArrayList.get(index)`

Because the elements are stored in an array:

```
list.get(500);
```

essentially performs:

```
array[500]
```

Therefore:

```
get(index) → 0(1)  
set(index) → 0(1)
```

The Java API explicitly documents constant-time positional access for `ArrayList`. ([GitHub](#))

6. `ArrayList` Insert in the Middle

Suppose:

```
[A][B][C][D][E]
```

You do:

```
list.add(2, "X");
```

The elements after index 2 must shift:

```
[A][B][C][D][E]  
      ↓  
[A][B][X][C][D][E]
```

So:

```
insert middle → 0(n)  
remove middle → 0(n)
```

Because elements have to be shifted.

7. Why `ArrayList` Is Usually Better Than `LinkedList`

For most ordinary application code:

```
ArrayList
→ contiguous array
→ excellent cache locality
→ O(1) random access
→ generally lower per-element overhead
```

LinkedList:

```
Node → Node → Node → Node
```

Each node contains links to other nodes.

Even though linked-list insertion/removal can be O(1) **after you already have the node/position**, finding an arbitrary position is O(n).

Don't say:

```
"LinkedList insertion is always O(1)."
```

That's incomplete.

8. ArrayList Interview Trap: Thread Safety

`ArrayList` is **not synchronized**. If multiple threads structurally modify it concurrently, external synchronization is required. ([GitHub](#))

This:

```
List<Integer> list =
    new ArrayList<>();
```

doesn't make it thread-safe.

Possible alternatives:

```
Collections.synchronizedList(...)
```

or:

```
CopyOnWriteArrayList
```

depending on the workload.

9. ArrayList Cheat Sheet

Operation	Typical complexity
<code>get(index)</code>	O(1)
<code>set(index)</code>	O(1)
<code>add(element)</code>	O(1) amortized
<code>add(index, element)</code>	O(n)
<code>remove(index)</code>	O(n)
<code>contains()</code>	O(n)

Search unsorted list	
----------------------	--

	O(n)
--	------

10. HashMap Internal Structure

Now the important one.

A `HashMap` is conceptually:

```
HashMap
  ↓
bucket array
  ↓
bucket
  ↓
Node(s)
```

Example:

```
table
index
0 → null
1 → Node
2 → null
3 → Node → Node
4 → null
5 → Node
...
```

Each normal node conceptually contains:

```
static class Node<K,V> {
    int hash;
    K key;
    V value;
    Node<K,V> next;
}
```

The JDK implementation indeed uses nodes with hash, key, value and next references. ([GitHub](#))

11. HashMap `put()` — Step by Step

Suppose:

```
map.put("Avnish", 100);
```

Step 1: Calculate hash

Java starts from:

```
key.hashCode()
```

`HashMap` applies a hash transformation/spread so that useful hash bits influence bucket selection.

Conceptually:

```
"Avnish"
  ↓
hashCode()
  ↓
spread hash
```

Step 2: Find bucket

For a power-of-two table, the implementation uses:

```
index = (n - 1) & hash;
```

where:

```
n = table length
```

Instead of `%`.

For example:

```
table length = 16  
  
index = (16 - 1) & hash  
       = 15 & hash
```

The current OpenJDK implementation requires the table capacity to be a power of two and uses this style of indexing. ([GitHub](#))

12. Why Power of Two?

Suppose:

```
capacity = 16
```

Binary:

```
16 → 10000  
15 → 01111
```

Then:

```
hash & 01111
```

efficiently extracts the lower bits needed for the bucket index.

That's why HashMap capacities are power-of-two based.

13. Empty Bucket

If the bucket is empty:

```
bucket[5] = null
```

HashMap stores:

```
bucket[5]  
  ↓  
Node(key, value)
```

Done.

Expected:

```
0(1)
```

14. What if Two Keys Go to the Same Bucket?

That's a **collision**.

Suppose:

```
Key A → bucket 5  
Key B → bucket 5
```

Conceptually:

```
bucket[5]  
  ↓  
Node A → Node B
```

Historically this was represented using a linked list.

In modern Java, sufficiently large collision chains can be converted into a **red-black tree**. ([GitHub](#))

15. Why `hashCode()` Alone Is Not Enough

Suppose:

```
key1.hashCode() == key2.hashCode()
```

That doesn't mean:

```
key1.equals(key2)
```

is true.

HashMap ultimately compares both:

```
hash  
+  
equals()
```

Conceptually:

```
if (hash matches &&  
    (key == existingKey ||  
     key.equals(existingKey))) {  
    // same mapping  
}
```

That's why the `equals()` / `hashCode()` contract is critical for HashMap keys.

16. Collision Resolution

Modern HashMap uses:

```
Few collisions:  
bucket → linked list  
  
Heavy collisions:  
bucket → balanced tree
```

The OpenJDK implementation has:

```
TREEIFY_THRESHOLD = 8
UNTREEIFY_THRESHOLD = 6
MIN_TREEIFY_CAPACITY = 64
```

So a bin doesn't automatically become a tree just because there are eight entries; the table must also be sufficiently large. If the table is too small, resizing is preferred. ([GitHub](#))

This is an excellent interview nuance.

17. Why Red-Black Tree?

A linked list lookup can degrade toward:

```
O(n)
```

A balanced tree can provide roughly:

```
O(log n)
```

for lookup within a heavily-collided bin.

Thus Java 8 introduced tree bins to improve worst-case behavior for heavy hash collisions. ([GitHub](#))

But don't say:

```
“HashMap lookup is always O(log n).”
```

Normal well-distributed lookups are expected O(1).

18. HashMap `get()`

Suppose:

```
map.get("Avnish");
```

Conceptually:

```
key
↓
hashCode()
↓
hash spread
↓
bucket index
↓
bucket
↓
compare hash
↓
compare equals()
↓
value
```

If the bucket has:

```
one node
```

easy.

If linked list:

```
Node → Node → Node
```

search within it.

If tree:

```
Red-Black Tree
```

search tree.

19. HashMap Resize

Default Java 17 `HashMap` configuration is:

```
initial capacity = 16  
load factor = 0.75
```

The resize threshold is approximately:

```
capacity × load factor
```

So:

```
16 × 0.75 = 12
```

When the number of entries crosses the threshold, the table is resized. ([Oracle Docs](#))

Conceptually:

```
capacity = 16  
threshold = 12  
  
insert #13  
  ↓  
resize  
  ↓  
capacity ≈ 32
```

The JDK documentation describes the table as being rehashed/rebuilt to approximately twice the number of buckets when the load threshold is exceeded. ([Oracle Docs](#))

20. What Happens During Resize?

Suppose:

```
Old capacity = 16  
New capacity = 32
```

The entries must be redistributed between the new buckets.

This is one reason resizing is expensive:

```
O(n)
```

Therefore if you know approximately how many entries you'll store, supplying an appropriate initial capacity can reduce repeated resizes. The Java docs specifically recommend considering initial

capacity and expected size when many entries will be stored. ([Oracle Docs](#))

21. Why Load Factor 0.75?

It's a compromise:

```
Lower load factor
→ more memory
→ fewer collisions

Higher load factor
→ less memory
→ potentially more collisions
```

Java's default is:

```
0.75
```

The JDK docs describe it as a reasonable time/space trade-off. ([Oracle Docs](#))

22. HashMap Complexity

Expected:

```
put() → O(1)
get() → O(1)
remove() → O(1)
```

under good hash distribution.

Worst-case collision behavior is improved by tree bins, roughly:

```
O(log n)
```

within a treeified bin.

But overall performance depends heavily on the quality/distribution of `hashCode()`.

23. HashMap Null Behavior

HashMap allows:

```
map.put(null, value);
```

and multiple values can be null.

The Java 17 docs explicitly state that `HashMap` permits null keys and null values. ([Oracle Docs](#))

This becomes important when comparing it to `ConcurrentHashMap`.

24. Why Must HashMap Keys Be Immutable?

Suppose:

```
class Employee {
```

```
String id;  
}
```

and:

```
Employee e = new Employee("101");  
map.put(e, "Avnish");
```

Now change:

```
e.id = "999";
```

If `hashCode()` depends on `id`, the key's hash effectively changes after insertion.

The map may now look in the wrong bucket when searching for the same object.

So:

“ A key should have stable `equals()` / `hashCode()` behavior while it is being used as a map key.

Immutable keys are safest.

25. Now: ConcurrentHashMap

`ConcurrentHashMap` is designed for concurrent access.

Its goal:

```
Multiple threads  
  ↓  
read/write same map  
  ↓  
without one global lock blocking everything
```

The Java 17 API describes it as providing concurrent retrievals and high expected concurrency for updates; retrieval operations generally do not block. ([Oracle Docs](#))

26. Important: Java 7 vs Java 8+ ConcurrentHashMap

This is an **interview trap**.

Java 7-era explanation

People often say:

```
ConcurrentHashMap uses segments.
```

That was true for older implementations.

Java 8+

The implementation moved away from segment-based locking.

Modern Java uses a structure based around:

```
bucket array  
+  
CAS  
+
```

```
per-bin synchronization
+
tree bins
+
cooperative resizing
```

So for your Java 8/11/17 interview:

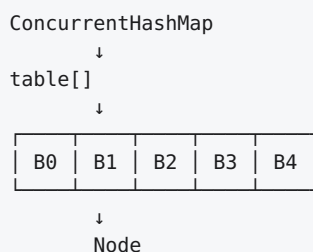
“ Do not explain modern `ConcurrentHashMap` as “16 segments.”

That is an outdated implementation model.

The current OpenJDK source describes bins, `Node`, `TreeBin`, `ForwardingNode`, and concurrent updates using fine-grained mechanisms. ([GitHub](#))

27. `ConcurrentHashMap` Internal Structure

Conceptually:



Similar to `HashMap`, bins can contain:

```
Node → Node → Node
```

or tree bins when collision chains become sufficiently large.

But operations are made concurrency-safe with finer-grained mechanisms.

28. How `get()` Works in `ConcurrentHashMap`

A major advantage:

```
get()
```

generally doesn't acquire a global lock.

Conceptually:

```
Thread A → get(key1)
Thread B → get(key2)
Thread C → get(key3)
```

can proceed concurrently.

The Java documentation explicitly states that retrieval operations generally do not block and can overlap with updates. ([Oracle Docs](#))

29. How `put()` Works in `ConcurrentHashMap`

Simplified conceptually:

```
put(key, value)
  ↓
calculate hash
  ↓
find bucket
  ↓
bucket empty?
  ├── yes → CAS insertion
  └── no
      ↓
coordinate with bin
  ↓
update/insert safely
```

For an existing/non-empty bin, the implementation uses fine-grained synchronization rather than locking the whole map. Modern OpenJDK source shows bin-level coordination and specialized nodes during updates/resizing. ([GitHub](#))

30. Why CAS?

CAS = **Compare-And-Swap**.

Conceptually:

```
Current bucket = null

Thread A:
"If bucket is still null,
place my node here."

Thread B:
"If bucket is still null,
place my node here."
```

Only one succeeds.

The other thread detects that the state changed and retries/continues through the appropriate update path.

This allows certain operations to avoid a large synchronized critical section.

31. Does ConcurrentHashMap Use Locks?

Yes — but **not one global lock for the whole map**.

A simplified modern mental model:

```
Empty bucket
  ↓
CAS

Occupied bucket
  ↓
fine-grained synchronization around that bin/update

Tree bin
  ↓
tree-specific coordination
```

This is much better than saying:

“

“ConcurrentHashMap is completely lock-free.”

It isn't.

32. Why ConcurrentHashMap Scales Better Than Hashtable

Imagine:

```
Hashtable
```

and:

```
Thread A → put()  
Thread B → get()  
Thread C → put()
```

Hashtable synchronizes its operations, so threads contend on the same object monitor.

Modern ConcurrentHashMap allows retrievals without locking and uses fine-grained coordination for updates, giving much greater concurrency. ([Oracle Docs](#))

33. Hashtable Internal Working

`Hashtable` is an older hash-table implementation.

It also uses:

```
bucket array  
+  
entries  
+  
hashing  
+  
collision chains
```

But its methods are synchronized.

The Java 17 API explicitly states:

“

Hashtable is synchronized.

It also recommends `ConcurrentHashMap` when a highly concurrent thread-safe map is desired. ([Oracle Docs](#))

34. Why Hashtable Is Slower Under Concurrency

Suppose:

```
Thread A → put()  
Thread B → get()  
Thread C → remove()
```

Because the map operations are synchronized, access can contend on the same object-level

synchronization mechanism.

So:

```
Hashtable
  ↓
coarse-grained synchronization
  ↓
more contention
```

while:

```
ConcurrentHashMap
  ↓
fine-grained concurrency
  ↓
better scalability
```

This is the key conceptual difference.

35. `Hashtable` VS `ConcurrentHashMap`

Feature	<code>Hashtable</code>	<code>ConcurrentHashMap</code>
Thread-safe	✓	✓
Synchronization	Coarse-grained synchronized methods	Fine-grained concurrency mechanisms
Concurrent reads	More contention	Generally non-blocking
Null key	✗	✗
Null value	✗	✗
Modern choice	Usually no	✓
Iterator behavior	Fail-fast collection iterators	Weakly consistent
High concurrency	Poorer scalability	Designed for high concurrency

Oracle explicitly recommends `ConcurrentHashMap` over `Hashtable` for highly concurrent thread-safe use. ([Oracle Docs](#))

36. Iterator Difference

`Hashtable`

Its collection-view iterators are fail-fast:

```
structural modification during iteration
  ↓
ConcurrentModificationException
```

The Java 17 docs describe its iterators as fail-fast on structural modification, on a best-effort basis. ([Oracle Docs](#))

`ConcurrentHashMap`

Its iterators do **not** throw `ConcurrentModificationException` merely because the map is concurrently updated.

Instead, they are **weakly consistent**.

Meaning approximately:

```
Iterator starts
  ↓
Other thread modifies map
  ↓
Iterator continues safely
  ↓
May see some updates, but not necessarily all
```

This is intentionally different from a fail-fast iterator. ([Oracle Docs](#))

37. Why Doesn't ConcurrentHashMap Allow `null`?

Both:

```
Hashtable → no null
ConcurrentHashMap → no null
```

For ConcurrentHashMap, `null` has a useful semantic role: because keys and values cannot be null, a null result from `get()` can reliably indicate that no mapping is currently present. ([Oracle Docs](#))

Example:

```
map.get("A")
```

returns:

```
null
```

which unambiguously means no mapping.

With HashMap:

```
map.put("A", null);
```

Now:

```
map.get("A")
```

also returns null.

So absence and mapped-to-null are distinguishable only through another operation such as `containsKey()`.

38. `putIfAbsent()` — Very Important

ConcurrentHashMap provides atomic compound operations.

Instead of unsafe:

```
if (!map.containsKey(key)) {
    map.put(key, value);
}
```

use:

```
map.putIfAbsent(key, value);
```

Why?

Because the first version has a race:

```
Thread A → containsKey? false
Thread B → containsKey? false
Thread A → put
Thread B → put
```

`putIfAbsent()` is designed as one atomic map operation.

39. `computeIfAbsent()`

Extremely useful:

```
ConcurrentHashMap<String, Integer> map =
    new ConcurrentHashMap<>();

map.computeIfAbsent(
    "PAYMENT",
    key -> calculateInitialValue(key)
);
```

The computation/insertion behavior is coordinated atomically for the key.

A common real-world pattern:

```
ConcurrentHashMap<String, LongAdder> counters =
    new ConcurrentHashMap<>();

counters
    .computeIfAbsent("PAID", k -> new LongAdder())
    .increment();
```

This is directly documented as a scalable frequency-map pattern by the JDK. ([Oracle Docs](#))

40. Thread-Safe Map Does Not Mean Thread-Safe Value

This is an **excellent interview trap**.

Suppose:

```
ConcurrentHashMap<String, ArrayList<Integer>> map;
```

The map operations are thread-safe.

But:

```
map.get("A").add(10);
```

may still be unsafe because the `ArrayList` itself isn't thread-safe.

So:

```
ConcurrentHashMap
    ≠
    everything stored inside is thread-safe
```

The thread-safety guarantee applies to the map's operations, not arbitrary mutable objects stored as values.

41. ConcurrentHashMap vs `Collections.synchronizedMap()`

Another common question.

```
Map<K,V> map =  
    Collections.synchronizedMap(new HashMap<>());
```

makes the map thread-safe by wrapping operations with synchronization.

Conceptually:

```
all access  
  ↓  
one synchronization mechanism
```

`ConcurrentHashMap` is specifically designed for greater concurrency, especially with concurrent reads and updates.

So:

```
synchronizedMap  
→ simpler coarse-grained synchronization  
  
ConcurrentHashMap  
→ purpose-built concurrent map
```

42. HashMap vs ConcurrentHashMap

Feature	HashMap	ConcurrentHashMap
Thread-safe	✗	✓
Null key	✓	✗
Null value	✓	✗
Concurrent access	Unsafe for structural modification	Designed for it
Reads	Fast in single-threaded use	Highly concurrent
Iterators	Fail-fast, best effort	Weakly consistent
Atomic operations	Basic Map operations	<code>putIfAbsent</code> , <code>compute</code> , <code>merge</code> , etc.
Typical use	Normal map	Shared concurrent map

`HashMap` itself explicitly requires external synchronization when structurally modified by multiple threads. ([Oracle Docs](#))

43. Hashtable vs HashMap

This question is older but still common.

Feature	HashMap	Hashtable
Thread-safe	✗	✓
Null key	✓	✗
Null value	✓	✗

Synchronization	None	Synchronized
Modern choice	✓	Usually no
Introduced	Java 1.2 Collections	Java 1.0

The Java 17 docs describe HashMap as roughly equivalent to Hashtable except that HashMap is unsynchronized and permits nulls. ([Oracle Docs](#))

44. One Diagram You Should Memorize

ArrayList

```

ArrayList
  ↓
Object[]
  ↓
[A][B][C][D][ ][ ]
  ↓
get(index)
  ↓
direct array access
  ↓
O(1)

```

HashMap

```

Key
  ↓
hashCode()
  ↓
spread hash
  ↓
bucket index
  ↓
┌ bucket
│ Node → Node
└
  ↓
collision
  ↓
treeify when appropriate

```

ConcurrentHashMap

```

Thread A ┌
Thread B ┤ → ConcurrentHashMap
Thread C └
  ↓
table / bins
  ↓
CAS + fine-grained
synchronization

```

45. Interview Scenario: HashMap

INTERVIEW SCENARIO

Interviewer:

“What happens when you call `map.put(key, value)` ?”

Answer:

1. HashMap gets `key.hashCode()`
2. Hash is spread/transformed.
3. Bucket index is calculated from hash and table length.
4. If bucket is empty, a node is inserted.
5. If bucket contains entries, hashes and keys are compared.
6. If the key already exists, its value is replaced.
7. Otherwise a new node is added to the collision structure.
8. Large collision bins may become red-black trees.
9. If size crosses the resize threshold, the table grows and entries are redistributed.

That is the answer to memorize.

46. Interview Scenario: Why Is HashMap `get()` $O(1)$?

INTERVIEW SCENARIO

Good answer:

“HashMap uses the key’s hash to directly determine a bucket, so under a good hash distribution it can find the relevant bucket in constant expected time. It then searches only the entries within that bucket. Java 8+ can convert heavily collided buckets into tree bins, improving collision-heavy lookup behavior.

The official Java docs describe basic `get / put` as constant-time assuming proper hash dispersion. ([Oracle Docs](#))

47. Interview Scenario: Why ConcurrentHashMap Over Hashtable?

INTERVIEW SCENARIO

A strong answer:

“Hashtable achieves thread safety through synchronized operations, which creates coarse-grained contention. ConcurrentHashMap is designed for high concurrency: retrievals generally don’t block and updates use finer-grained coordination rather than synchronizing the entire map. It also provides atomic operations such as `putIfAbsent` and `computeIfAbsent`, making it more suitable for shared mutable maps in concurrent applications. ([Oracle Docs](#))

48. The Most Important Traps

△ COMMON TRAP

Trap 1

“*ConcurrentHashMap uses segments.*”

For Java 8+ modern implementation: outdated.

Use:

```
CAS
+
bin-level synchronization
+
tree bins
+
cooperative resizing
```

instead. ([GitHub](#))

△ COMMON TRAP

Trap 2

“*HashMap is $O(\log n)$.*”

No.

Expected:

```
O(1)
```

with good hashing.

Heavy collision bins can become trees.

△ COMMON TRAP

Trap 3

“*ArrayList capacity is always 10.*”

Not an API guarantee.

In Java 17, `new ArrayList()` is documented as an empty list with an initial capacity of 10, while the underlying growth policy remains implementation-specific. ([Oracle Docs](#))

△ COMMON TRAP

Trap 4

“*ConcurrentHashMap makes compound business logic atomic.*”

No.

This:

```
if (!map.containsKey(k)) {
    map.put(k, v);
}
```

```
}
```

is still not atomic as a combined sequence.

Use an atomic map operation:

```
map.putIfAbsent(k, v);
```

where appropriate.

⚠ COMMON TRAP

Trap 5

“*Hashtable is the best thread-safe HashMap.*”

No.

For modern concurrent applications:

```
ConcurrentHashMap
```

is generally the appropriate choice when you need a highly concurrent map. Oracle's Java 17 docs explicitly recommend it over Hashtable for highly concurrent thread-safe use. ([Oracle Docs](#))

49. Final Interview Cheat Sheet

ARRAYLIST

Backing structure	→ dynamic array
get	→ O(1)
set	→ O(1)
append	→ O(1) amortized
middle insert	→ O(n)
middle remove	→ O(n)
thread-safe	→ No

HASHMAP

Structure	→ bucket array + nodes
Hash	→ hashCode() + hash spreading
Index	→ (n - 1) & hash
Collision	→ linked nodes / tree bin
Treeification	→ threshold 8, table capacity >= 64
Default capacity	→ 16
Load factor	→ 0.75
get/put expected	→ O(1)
Thread-safe	→ No
Null key/value	→ Allowed

CONCURRENTHASHMAP

Thread-safe	→ Yes
Null key/value	→ No
get	→ generally non-blocking
Updates	→ fine-grained concurrency
Mechanisms	→ CAS + bin coordination + tree bins
Atomic operations	→ putIfAbsent / compute / merge
Iterator	→ weakly consistent

Best use → shared concurrent map

HASHTABLE

Thread-safe	→ Yes
Synchronization	→ coarse-grained
Null key/value	→ No
Iterator	→ fail-fast
Legacy	→ Yes
Modern choice	→ usually ConcurrentHashMap

The one comparison to remember

```
HashMap
↓
Fast, simple, not thread-safe

Hashtable
↓
Thread-safe, coarse synchronization, legacy

ConcurrentHashMap
↓
Thread-safe, high concurrency, modern
```

For your interview preparation, the next natural step after this is `equals()` and `hashCode()` **contract** + **why HashMap breaks when a key is mutable**, because that is the question interviewers commonly ask immediately after explaining HashMap internals.